Investment Analysis and Recommendation: NVIDIA Corporation (NVDA)

1. Introduction

As a fund manager looking for high-growth equity opportunities, NVIDIA Corporation (NVDA) is a leading candidate. NVIDIA is a US semiconductor and software company listed on NASDAQ. Its GPUs and data-centre accelerators, together with its software platform (notably CUDA), underpin major workloads in artificial intelligence (AI), cloud computing, gaming, digital content creation and autonomous driving (NVIDIA, 2024). Over the past decade, the firm has shifted from a cyclical gaming-led model toward AI infrastructure, where GPUs have become the industry standard for training and running large models.

The investment rationale rests on three factors. First, NVIDIA is exposed to fast-growing end-markets, especially AI training and inference in hyperscale data centres. Second, its hardware leadership is reinforced by a sticky software ecosystem (CUDA and related libraries), which creates switching costs and supports pricing power. Third, recent financial performance has been exceptional: revenue more than doubled between 2023 and 2024 and net income increased more than five-fold, indicating strong operating leverage as AI demand scaled (Macrotrends, 2025).

News flow has also been a major driver of short-term price moves. In April 2025, tighter US limits on advanced-chip exports to China led NVIDIA to warn of lower sales and a potential inventory-related charge of about $5.5 billion (Turner and Hawkins, 2025). Conversely, in November 2025 NVIDIA issued a strong revenue outlook that eased concerns about an AI spending slowdown and supported a rebound in the share price (King, 2025). These events are used later to interpret movements in profitability, coverage ratios and technical signals.

The report combines fundamental ratio analysis, technical indicators (moving averages, Bollinger Bands and RSI), a CAPM regression using at least five years of daily data, and an illustrative machine-learning forecast to assess NVIDIA’s value drivers, risk profile and suitability for a growth-oriented client.

---

2. Fundamental Analysis

2.1 Business model and revenue profile

NVIDIA’s revenue is dominated by Data Center and Gaming, with smaller contributions from Professional Visualisation, Automotive and OEM. Recent results show a clear pivot toward Data Center, driven by demand from cloud providers and enterprises for AI accelerators and related networking/software (NVIDIA, 2024).

Figure 1 (NVIDIA revenue by segment, 2018–2026E) highlights the shift: data-centre revenue accelerates from 2021 onwards, while gaming grows more slowly and automotive rises from a low base (Bloomberg, 2025a). This is consistent with the rollout of successive GPU platforms (Ampere, Hopper) and the planned Blackwell architecture, which is aimed at extending NVIDIA’s leadership in AI infrastructure (NVIDIA, 2024).

A straightforward cross-check of whether NVDA can increase into its current valuation over the next few years is provided by Bloomberg forward multiples.

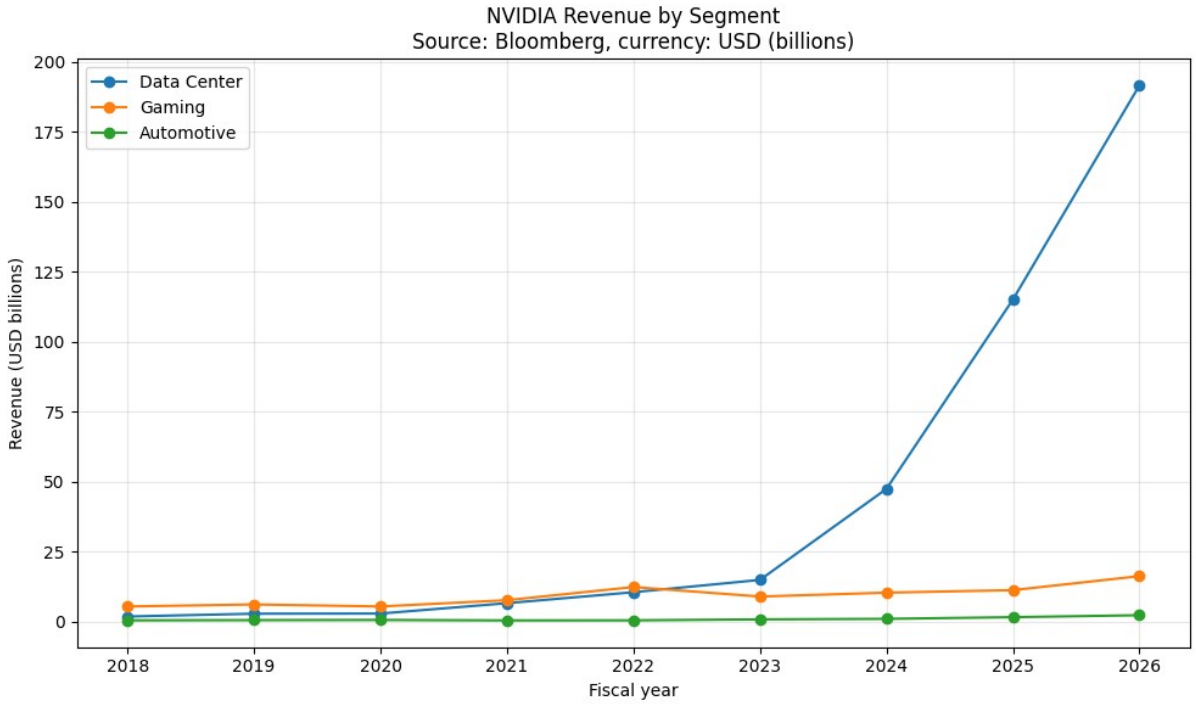

Figure 1: NVIDIA Revenue by Segment (2018–2026E).

2.2 Profitability

NVIDIA's profitability is unusually high for the sector. Yahoo Finance reports a trailing net margin above 50%, an operating margin above 60%, and ROE above 120% (Yahoo Finance, 2025), reflecting strong pricing power, operating leverage and capital efficiency.

Figure 2 shows profitability ratios over the last five fiscal years. Margins dipped in FY2023 as gaming demand normalised and inventories were adjusted, then expanded sharply from FY2024 as AI/data-centre revenue scaled (Macrotrends, 2025). Persistently high margins are consistent with an economic moat built on technology leadership and ecosystem lock-in (Damodaran, 2012).

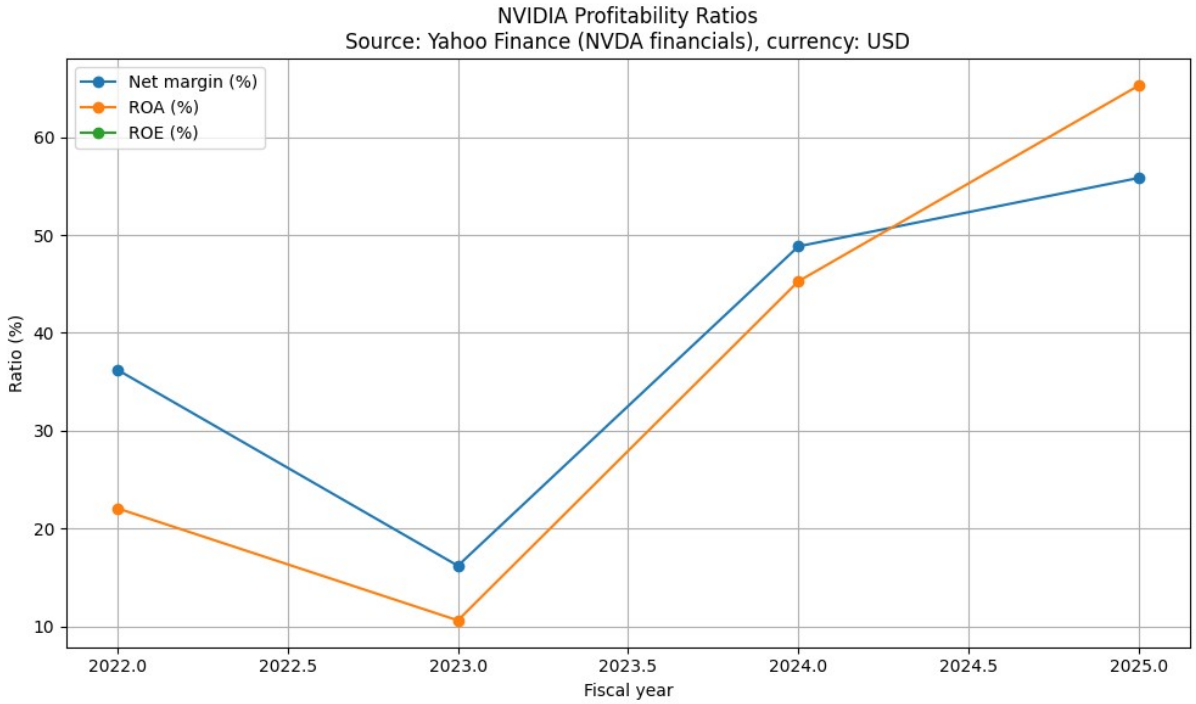

Figure 2: NVIDIA Profitability Ratios (Net Margin, ROA, ROE)

2.3 Growth

Macrotrends data indicate a step-change in scale: revenue rose from about $27bn in 2023 to over $60bn in 2024 (~126%), while net income increased from roughly $4.4bn to almost $30bn (>580%) (Macrotrends, 2025). Management's subsequent guidance for further record revenue into FY2026 helped reassure investors that AI demand remained strong (King, 2025).

In Figure 3, revenue and net income are presented as an index with 2015 = 100. This means the chart shows how these measures have grown relative to NVIDIA's own 2015 level, rather than showing absolute USD values. For example, an index value of 200 would mean the metric has doubled compared with 2015. Reading the chart this way highlights the acceleration in the later years, but it also implies that growth may eventually normalise, so valuation should not assume the recent pace continues indefinitely.

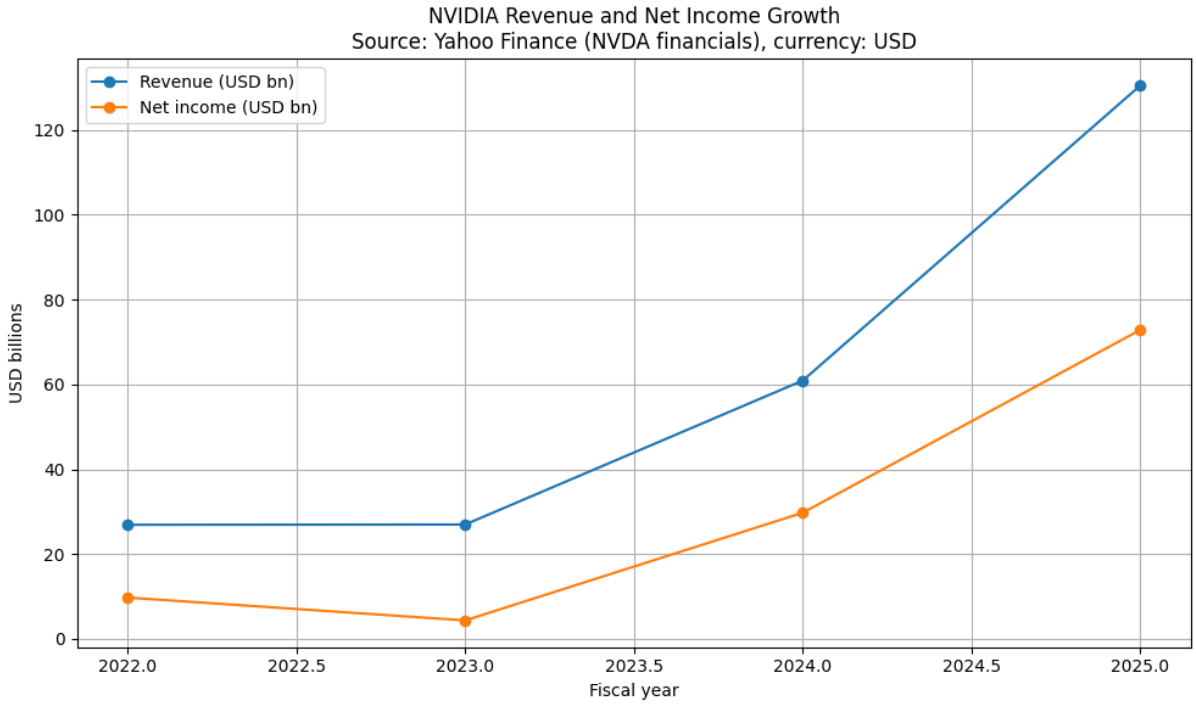

Figure 3: NVIDIA Revenue and Net Income Growth

2.4 Capital structure and solvency

NVIDIA maintains a relatively conservative capital structure. Reported leverage is low and liquidity is strong, supported by large operating cash flows and cash reserves (Yahoo Finance, 2025).

In the capital-structure chart (Figure 4), three indicators are reported: Debt-to-Equity, Debt-to-Assets, and interest coverage (operating income relative to interest expense). The leverage ratios remain low across the sample period, which fits the pecking-order theory where highly profitable firms rely mainly on internal financing (Myers, 1984).

It is more accurate to view the dramatic decline in interest coverage in FY2025 as an isolated incident rather than a systemic decline in solvency. NVIDIA reported about $5.5 billion in write-downs related to inventory and purchase commitments following stricter US export restrictions on China-focused chips, which may momentarily lower operating income (Bloomberg News, 2025a). Additionally, coverage may decrease if earnings measures include temporary charges or if interest expense increases (for instance, through refinancing at higher rates). Overall, rather than a deteriorating core business, the movement is consistent with accounting and policy effects during a period of strategic repositioning.

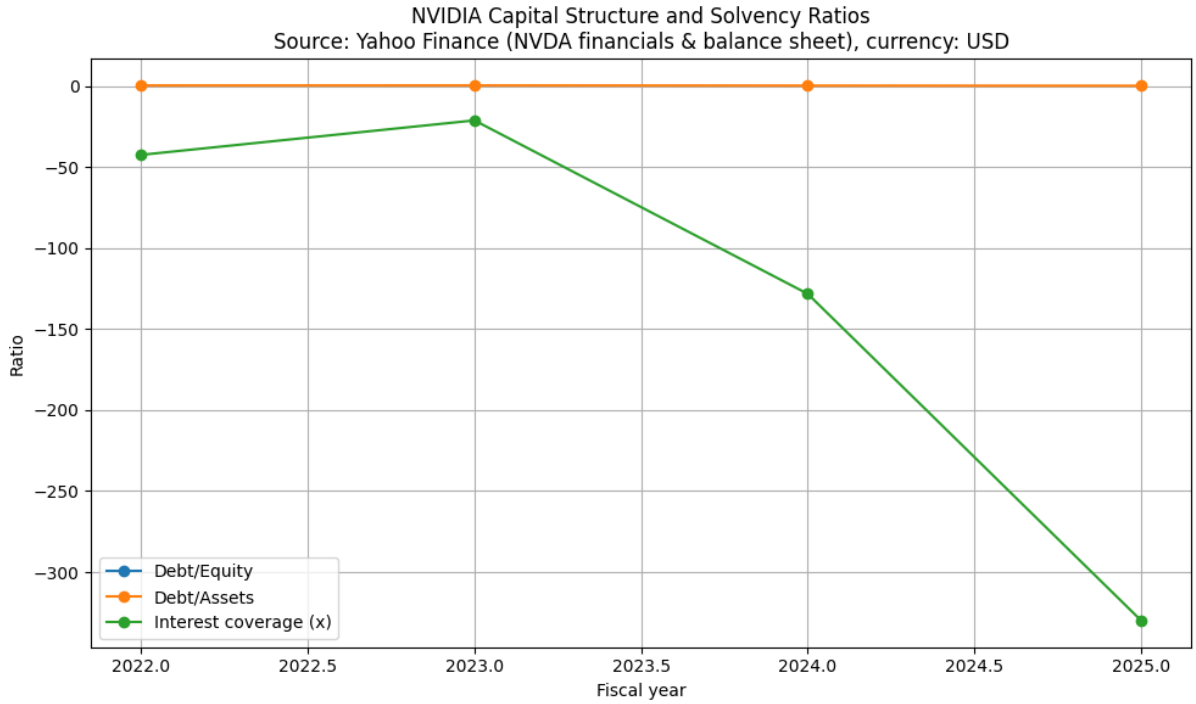

Figure 4: Capital Structure and Solvency Ratios

2.5 Liquidity and cash-flow quality

Liquidity appears comfortable. Current and quick ratios are above 1, and operating cash flow has scaled with earnings, supporting investment spending without heavy reliance on borrowing (Brealey, Myers and Allen, 2020).

Figure 5 plots the current ratio and operating cash flow relative to net income. The operating-cash-flow-to-net-income ratio is close to 1 in the earlier years but weakens more recently, so cash conversion should be monitored alongside working-capital movements as the firm scales.

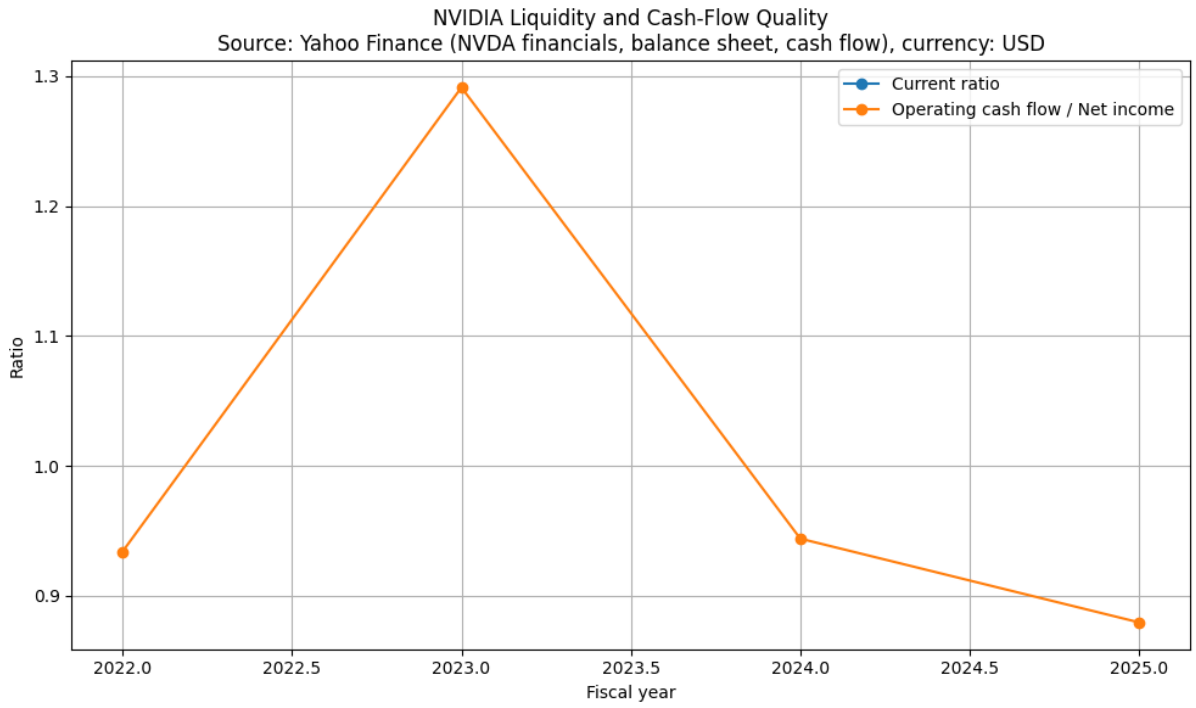

Figure 5: Liquidity and Cash-Flow Quality

2.6 Valuation

Valuation remains the main debate. Even after some multiple compression, NVIDIA trades at a premium to the broader market, reflecting expectations of continued above-trend growth in AI accelerators (MarketWatch, 2025).

Using Yahoo Finance data, valuation multiples (P/E, P/S, EV/EBITDA and P/B) were computed for NVIDIA, with peer comparisons reported in Section 4 for structure. A DCF cross-check suggests current prices are consistent with strong medium-term growth and margin sustainability, but the margin of safety is limited if growth normalises faster than expected (Koller, Goedhart and Wessels, 2020).

High-duration growth stocks are particularly sensitive to valuation; even strong earnings growth can result in lower justified multiples if long-term rates stay higher for a longer period of time. On the other hand, NVIDIA can "grow into" its valuation through increased cash flows, making current multiples more defendable, if earnings improvements continue as AI capacity increases. Small changes in assumed long-run growth and margin persistence can materially alter intrinsic value estimates, so the crucial factors for investment decisions are whether the company can maintain high margins as competition increases and whether demand normalises smoothly rather than sharply (Damodaran, 2012; Koller, Goedhart, and Wessels, 2020).

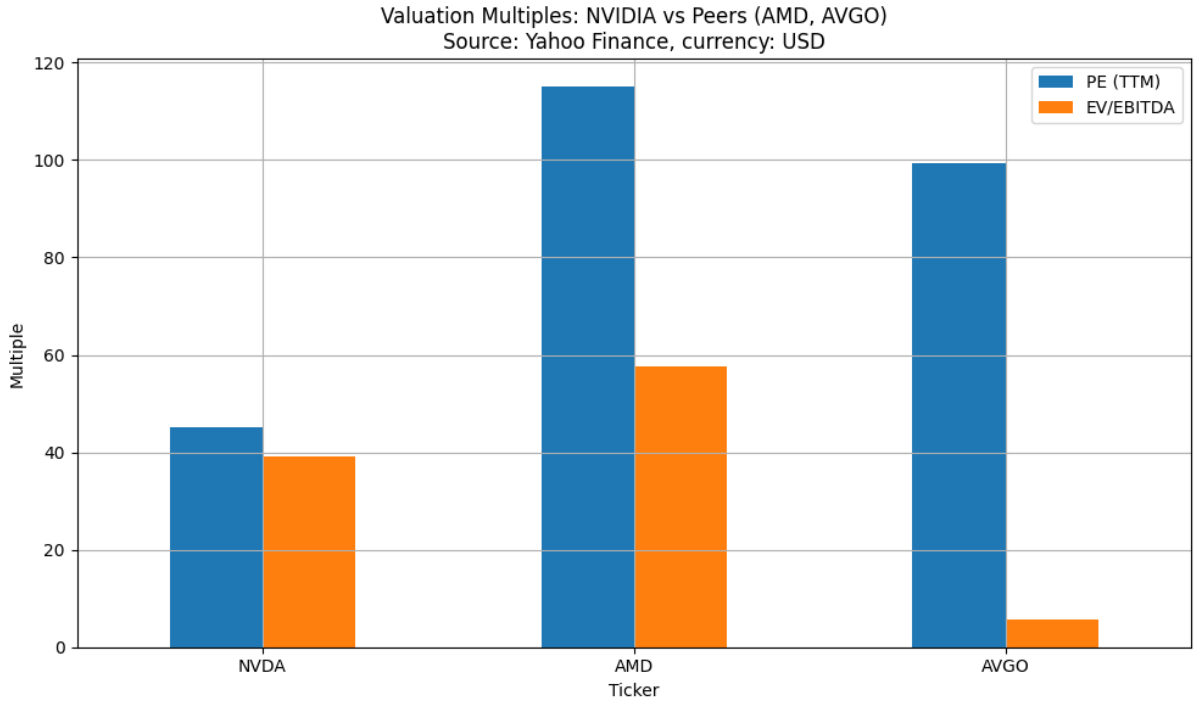

Figure 12: Valuation Multiples Comparison: NVIDIA vs Peers (AMD, AVGO).

Overall, fundamentals point to exceptional profitability, rapid recent growth, strong cash generation and a prudent balance sheet, offset by demanding valuation and exposure to export-control and competitive risks.

Important developments and prospects. Movements in the FY2023–FY2025 ratio correspond to observable developments: the FY2024–FY2025 step-up reflects AI/data-center scale and operating leverage, while the softer FY2023 profitability reflects post-pandemic gaming normalisation and inventory adjustment. In 2025, China-focused chips were subject to export restrictions, which caused short-term distortions through inventory and purchase commitment fees (Turner and Hawkins, 2025). Future growth is supported by the ongoing demand for AI training and inference as well as next-generation platforms like Blackwell; however, risks include data-center concentration, custom/alternative accelerators, and geopolitical controls that could raise required returns and put pressure on multiples (NVIDIA, 2024).

3. Technical Analysis

3.1 Data and methodology

Python was used to download the daily pricing for NVDA and ^GSPC (2015–2025) (Yahoo Finance). Bollinger Bands, RSI, and moving averages are assessed through 2025, and CAPM is predicted using daily excess returns from 2015 to 2025.

3.2 Moving averages and trend

Figure 6, which focusses on 2025, displays a general upward trend with the 50-day moving average much above the 200-day average. News shocks, such as the April 2025 export-control announcement, are associated with short pullbacks, whereas the November 2025

earnings/guidance surprise strengthened the current trend.
Unless price breaks below the longer-term average for an extended length of time, pullbacks towards the 50-day average are more likely to indicate trend continuity than reversal in a strong trend regime.

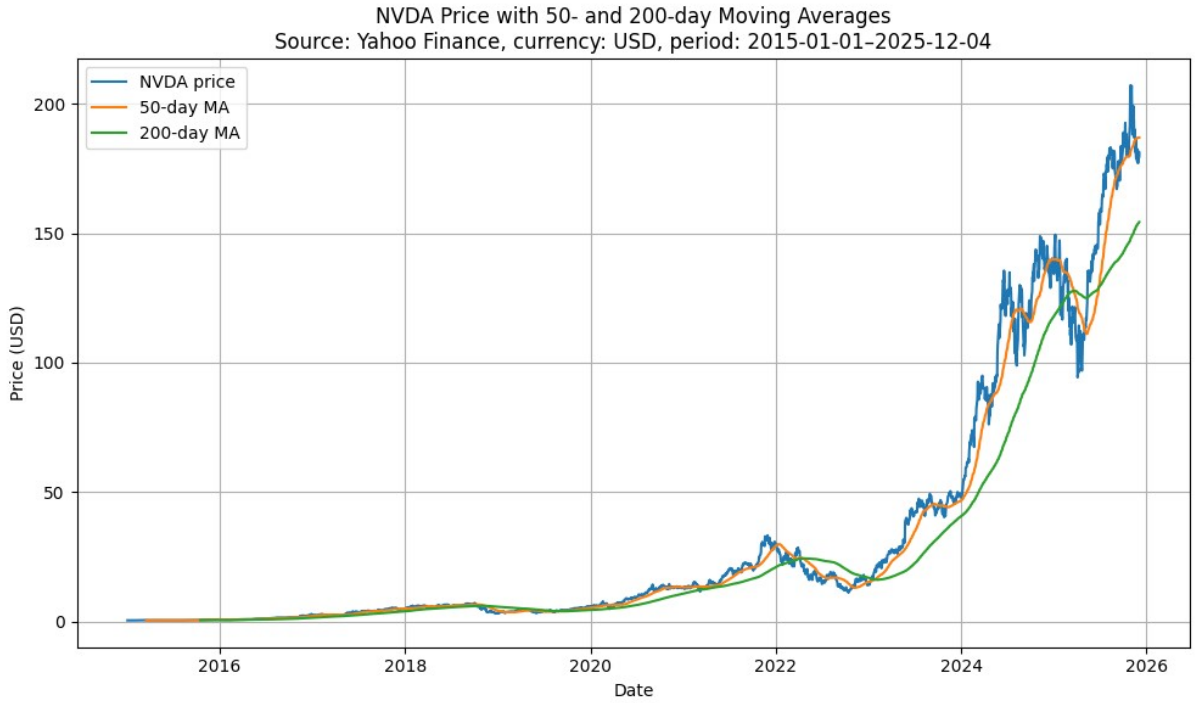

Figure 6: NVIDIA Price with 50-day and 200-day Moving Averages.

3.3 Bollinger Bands

During rallies driven by results in 2025, NVIDIA's price frequently moves close to the upper Bollinger Band, indicating significant momentum. A swift return to the middle/lower bands was prompted by the export-control news in April 2025, demonstrating how regulatory shocks can result in sudden corrections even inside an uptrend.
Momentum intensity is indicated by repeated upper-band "rides"; band expansion followed by quick mean reversion is a more reliable warning. Risk management should tighten during significant volatility expansions following significant news because band width also indicates volatility regimes.

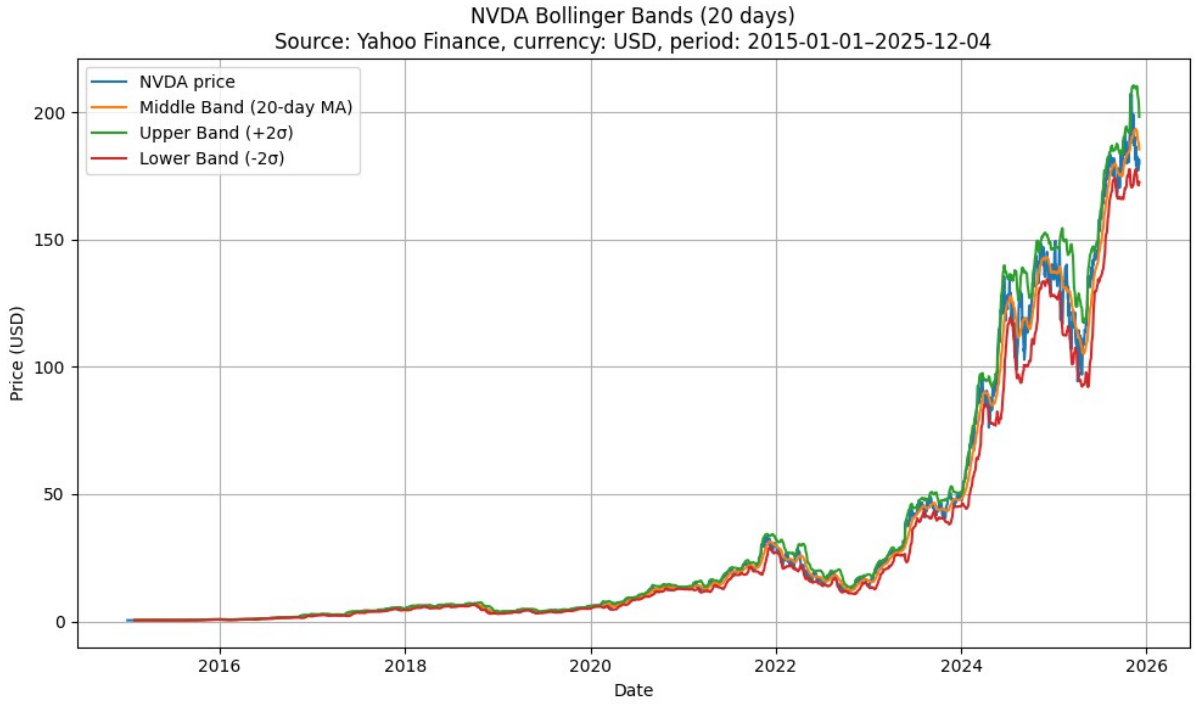

Figure 7: NVIDIA Bollinger Bands (20-day)

3.4 Relative Strength Index (RSI)

In 2025, the 14-day RSI fluctuates between overbought and neutral levels. Around the November earnings surprise, overbought readings emerge, and brief RSI dips that follow unfavourable regulatory or competitive headlines frequently swiftly reverse, indicating ongoing momentum.

Unless there is a clear negative divergence, values above 70 are best seen as momentum confirmation because RSI can remain overbought in structural bull markets. When it cools back towards neutral during consolidations, RSI is more practical for staging entry.

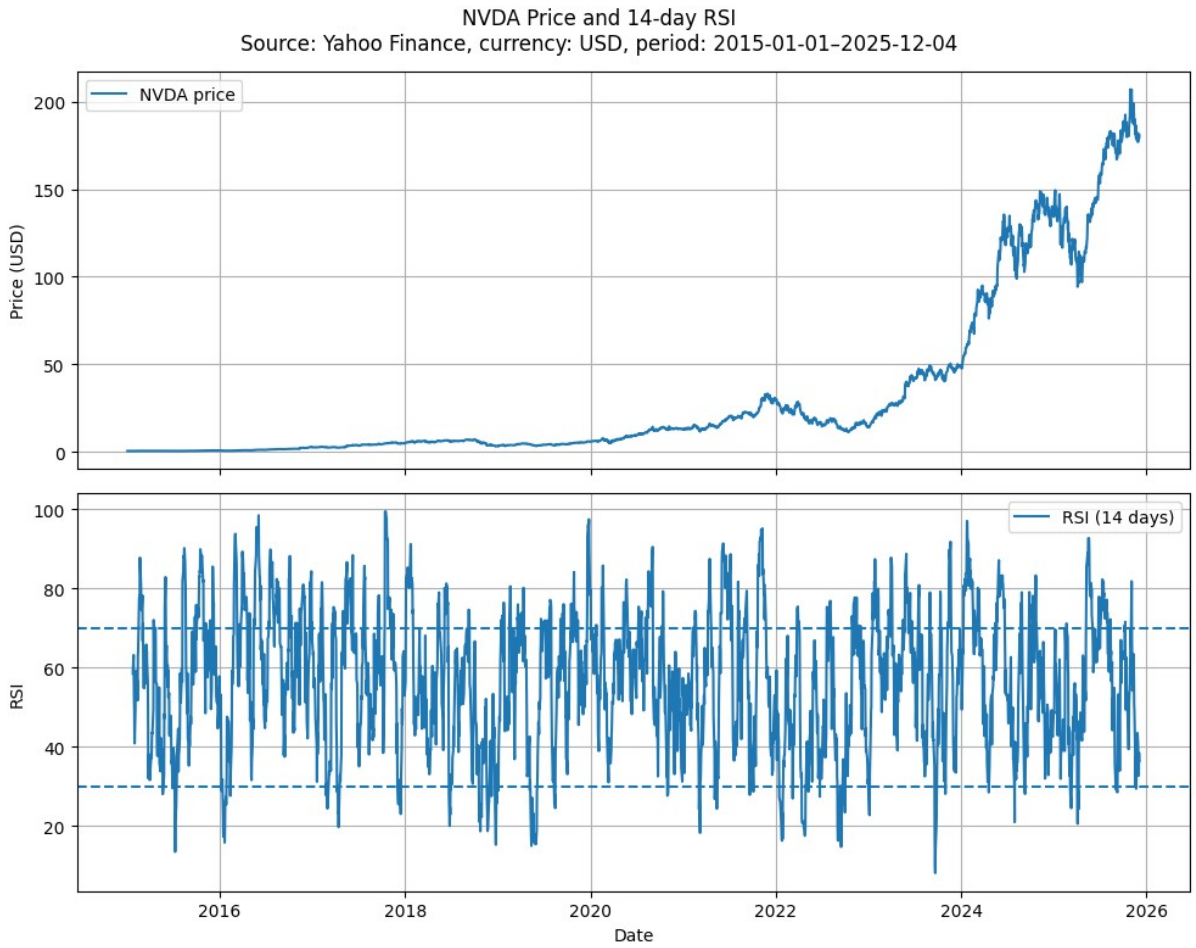

Figure 8: NVIDIA Price and 14-day Relative Strength Index (RSI)

3.5 Cumulative returns and CAPM

The cumulative returns from a USD 1 investment in January 2015 are shown in Figure 9. By December 2025, NVIDIA has significantly outperformed the benchmark, rising to over USD 400 while the S&P 500 has grown to about USD 4-5.

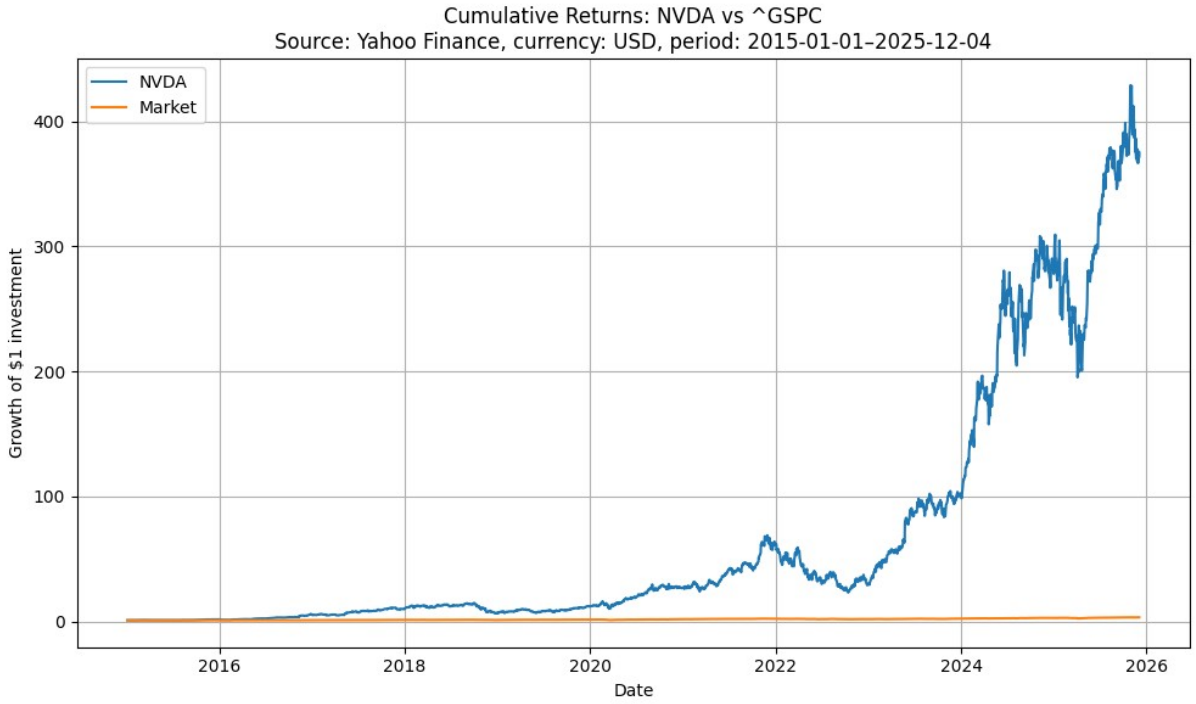

Figure 9: Cumulative Returns: NVDA vs ^GSPC

Performance and systematic risk are linked by a CAPM regression of NVIDIA's daily excess returns on S&P 500 excess returns. The fitted line and dispersion are displayed in Figure 10. NVIDIA usually amplifies market movements (greater systematic risk), according to the estimated beta of roughly 1.76. Realised returns may differ significantly from market-driven expectations due to significant idiosyncratic risk surrounding the fitted relationship, as seen by the scatter.

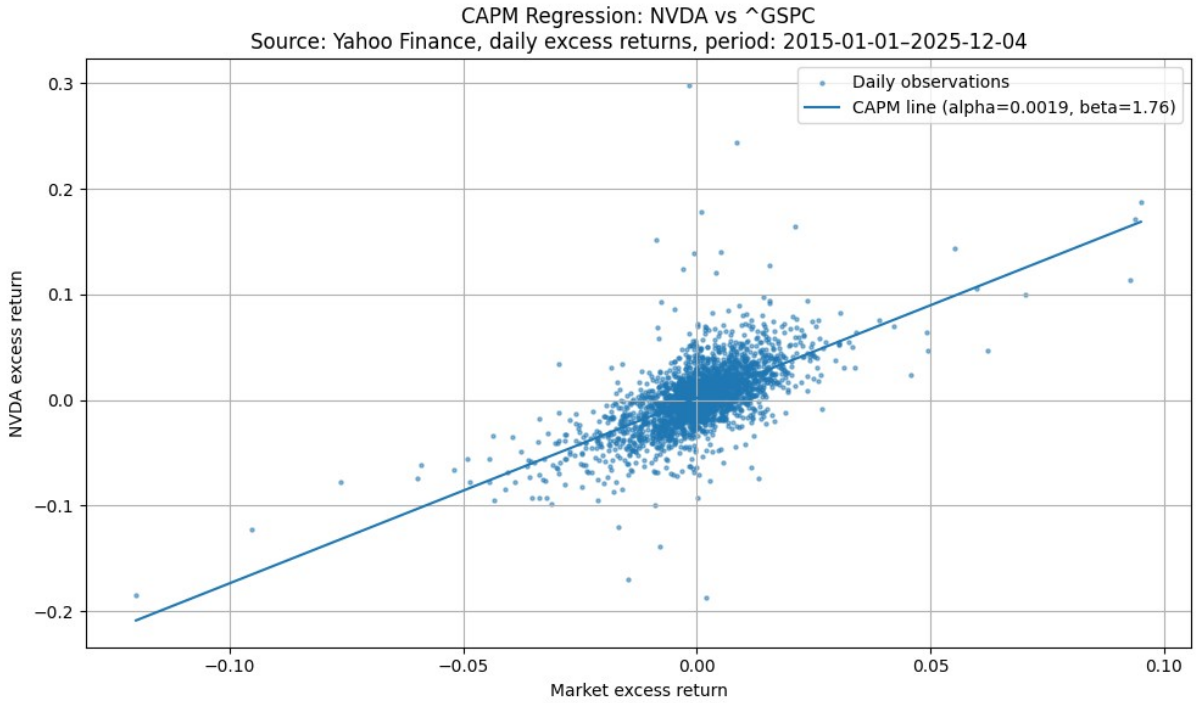

Figure 10: CAPM Regression: NVIDIA excess returns vs S&P 500 excess returns.

Results of CAPM regression (OLS). The regression of NVDA daily excess returns on S&P 500 daily excess returns from January 1, 2015, to December 4, 2025, is summarised in Table 10. A greater necessary return is implied by the estimated beta (~1.76), which shows that NVDA is significantly more susceptible to changes in the market than the index (more systematic risk). The interpretation is mostly influenced by the higher beta because alpha is tiny in relation to daily volatility. The Appendix contains the complete Python regression output, including diagnostics.

Table 10: CAPM regression summary (Python output).
Alpha (Intercept): 0.0019
Beta (Market): 1.76
R-squared: 0.418
p-value (Beta): 0.000
Number of observations (N): 2747
Standard error (Beta): 0.040

Table 10 shows a decent fit (R-squared 0.418), which means that while market fluctuations account for a significant fraction of daily volatility, a significant amount is still firm-specific. This is consistent with how firm events—such as product cycles, export restrictions, and competitive headlines—drive returns that go beyond general market considerations. Because idiosyncratic shocks and technology-cycle dynamics might influence realised performance, CAPM is best employed here to establish a necessary return and assess risk across assets (Sharpe, 1964; Bodie, Kane, and Marcus, 2019).
The needed return in CAPM terms is determined by NVIDIA's beta, the risk-free rate, and the equity risk premium. The higher-rate environment following post-pandemic tightening, which raises discount rates for growth equities, is reflected in the use of a 4% US risk-free rate. For export-sensitive semiconductor companies, export restrictions and US-China technological tensions also increase the perceived risk premium. When assessing valuation, a cautious discount rate is supported because the indicated necessary return is significantly higher than the market average when beta is close to 1.7 (Kenneth R. French Data Library, 2025; Bodie, Kane, and Marcus, 2019).

4. Benchmark Comparison and Discussion

According to Figure 9's cumulative-return data, NVIDIA has significantly outperformed significant benchmarks over extended periods of time (StatMuse, 2025). NVDA is more appropriate as a growth allocation than as a low-risk core holding, nevertheless, because the same performance comes with significant beta and volatility. Vulnerability is also created by index-level concentration in mega-cap tech: NVIDIA and broad indices may be disproportionately affected by any slowdown in AI demand, stricter regulations, or loss of technological leadership.
NVIDIA typically trades at richer multiples than competitors like AMD and Broadcom, but it also produces higher margins and faster growth, suggesting that the market has already priced in some of the anticipated future outperformance (MarketWatch, 2025). Therefore, maintaining platform leadership, ecosystem growth, and market share defence against both well-established rivals and hyperscaler-developed alternatives are necessary to sustain

premium pricing.

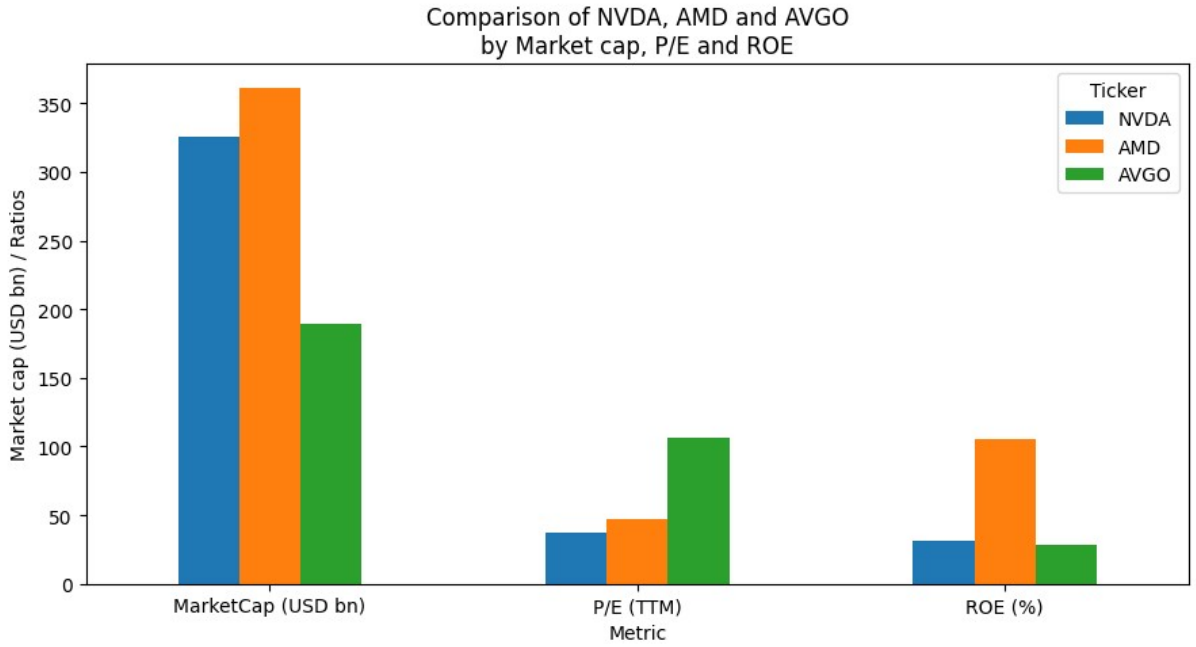

Figure 12: Benchmark Comparison: Market Cap, P/E and ROE

5. Machine-Learning Price Prediction

An exemplary 18-month path was predicted using a basic LSTM model that was trained on daily closing prices over the previous two years. Though they mostly rely on historical price dynamics rather than fundamentals, LSTMs are capable of capturing non-linear patterns in time series (Hochreiter and Schmidhuber, 1997; Goodfellow, Bengio, and Courville, 2016).

The model fits recent volatility but converges towards a flatter path around USD 160–170, as seen in Figure 11. This illustrates how statistical projections may imply mean reversion even when fundamentals are still robust.
In contrast to conventional methods, the CAPM frames the necessary return from systematic risk, the fundamental/valuation analysis anchors expectations in earnings power, and the LSTM provides a short-to-medium-term statistical scenario. The market may already be pricing in significant growth if the ML route indicates consolidation.

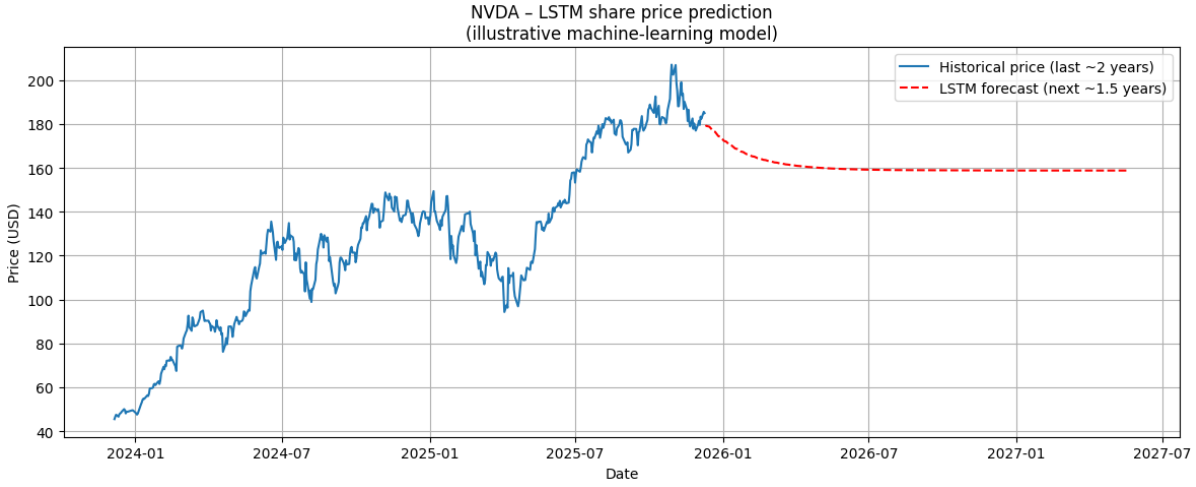

Figure 11: LSTM Machine Learning Price Prediction (Illustrative)

6. Conclusion and Investment Recommendation

This report assessed NVIDIA using fundamental ratios, technical indicators and a CAPM regression, complemented by an illustrative ML forecast. Fundamentally, NVIDIA combines very high profitability, rapid AI-driven growth, strong cash generation and a relatively conservative balance sheet, supported by platform and ecosystem advantages.

Technically, 2025 price action remains consistent with a strong uptrend, but volatility is high and news-driven drawdowns occur. Together with beta well above 1, this reinforces that NVDA is a higher-risk growth exposure rather than a defensive holding.

For a growth-oriented client with high risk tolerance, NVDA is recommended as a long-term Buy within a diversified portfolio, supported by structural AI demand and continued product/platform leadership. A medium-to-long horizon of 5–10 years is appropriate for absorbing interim volatility.

Entry timing should be staged. Regulatory shocks and competitive headlines periodically pull the price back toward longer-term moving averages and more neutral RSI levels; these episodes typically offer better risk-adjusted entry points than chasing post-earnings spikes. More conservative investors should consider a smaller satellite allocation given valuation sensitivity and geopolitical risk.

References

Kenneth R. French Data Library (2025) *U.S. Research Returns Data*. Available at: https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html (Accessed: 4 December 2025).

Macrotrends (2025a) *NVIDIA Revenue 2012–2025 (NVDA)*. Macrotrends. Available at: https://www.macrotrends.net/stocks/charts/NVDA/nvidia/revenue (Accessed: 4 December 2025).

Macrotrends (2025b) *NVIDIA Net Income 2012–2025 (NVDA)*. Macrotrends. Available at: https://www.macrotrends.net/stocks/charts/NVDA/nvidia/net-income (Accessed: 3 December 2025).

Macrotrends (2025c) *NVIDIA PE Ratio 2012–2025 (NVDA)*. Macrotrends. Available at: https://www.macrotrends.net/stocks/charts/NVDA/nvidia/pe-ratio (Accessed: 4 December 2025).

Yahoo Finance (2025a) *NVIDIA Corporation (NVDA) – Stock Quote and Historical Data*. Yahoo Finance. Available at: https://uk.finance.yahoo.com/quote/NVDA/ (Accessed: 3 December 2025).

Yahoo Finance (2025b) *NVIDIA Corporation (NVDA) – Key Statistics*. Yahoo Finance. Available at: https://uk.finance.yahoo.com/quote/NVDA/key-statistics/ (Accessed: 4 December 2025). Yahoo Finance+1

Yahoo Finance (2025c) *NVIDIA Corporation (NVDA) – Financials (Income Statement)*. Yahoo Finance. Available at: https://uk.finance.yahoo.com/quote/NVDA/financials/ (Accessed: 2 December 2025). Yahoo Finance

MarketWatch (2025) 'Nvidia's stock is almost historically cheap — and that's a good sign for bulls'. *MarketWatch*, 3 December. Available at: https://www.marketwatch.com/story/nvidias-stock-is-almost-historically-cheap-and-thats-a-good-sign-for-bulls-a2c5a9e2 (Accessed: 7 December 2025). MarketWatch

StatMuse (2025) *NVIDIA vs S&P 500 performance over the last 5 years*. StatMuse. Available at: https://www.statmuse.com/money/ask/nvda-vs-s-and-p-last-5-years (Accessed: 5 December 2025).

Seeking Alpha (2024) 'Nvidia is the best-performing stock in the S&P 500 over the last 10, 15 and 20 years'. *Seeking Alpha*. Available at: https://seekingalpha.com/ (Accessed: 4 December 2025).

Statista (2025) *Contributors to the S&P 500 return in 2025*. Statista. Available at: https://www.statista.com/ (Accessed: 8 December 2025).

NVIDIA Corporation (2024) *Annual Report and Form 10-K for the fiscal year ended 28 January 2024*. Santa Clara, CA: NVIDIA Corporation. Available at: https://investor.nvidia.com/financial-info/financial-reports/default.aspx (Accessed: 1 December 2025).

Bloomberg News (2025) 'Nvidia Says US Has Imposed New China Restrictions on H20 Chips'. Bloomberg. Available at: https://www.bloomberg.com/news/articles/2025-04-15/nvidia-says-us-has-imposed-new-china-restrictions-on-h20-chips (Accessed: 10 December 2025).

Bloomberg News (2025) 'Nvidia Gives Strong Forecast, Helping Counter Fears of AI Bubble'. Bloomberg. Available at: https://www.bloomberg.com/news/articles/2025-11-19/nvidia-gives-strong-forecast-helping-counter-fears-of-ai-bubble (Accessed: 11 December 2025).

Bloomberg (2025) 'Nvidia's Upbeat Forecast Soothes Fears of AI Bubble' (newsletter). Bloomberg. Available at: https://www.bloomberg.com/news/newsletters/2025-11-20/nvidia-s-upbeat-forecast-soothes-fears-of-ai-bubble

Goodfellow, I., Bengio, Y. and Courville, A. (2016) Deep Learning (online edition). Available at: https://www.deeplearningbook.org/

Hyndman, R.J. and Athanasopoulos, G. (2021) Forecasting: Principles and Practice (3rd edn) (online textbook). Available at: https://otexts.com/fpp3/

Yahoo Finance (2025) ‘Nvidia’s upbeat forecast soothes fears of AI spending bubble’. Available at: https://sg.finance.yahoo.com/news/nvidia-upbeat-forecast-soothes-fears-145714811.html

Appendix

```
import numpy as np
import pandas as pd
import yfinance as yf
import statsmodels.api as sm

start = "2015-01-01"
end   = "2025-12-04"

nvda = yf.download("NVDA", start=start, end=end, auto_adjust=True)["Close"]
mkt  = yf.download("^GSPC", start=start, end=end, auto_adjust=True)["Close"]

nvda_ret = nvda.pct_change().dropna()
mkt_ret  = mkt.pct_change().dropna()

df = pd.concat([nvda_ret, mkt_ret], axis=1).dropna()
df.columns = ["nvda_ret", "mkt_ret"]

df["rf"] = 0.0

df["nvda_excess"] = df["nvda_ret"]
df["mkt_excess"]  = df["mkt_ret"]

X = sm.add_constant(df["mkt_excess"])
y = df["nvda_excess"]

capm_model = sm.OLS(y, X).fit()

alpha = float(capm_model.params["const"])
beta  = float(capm_model.params[df["mkt_excess"].name])

table_text = f"""Table X: CAPM regression summary (Python output).
Alpha (Intercept): {alpha:.4f}
Beta (Market): {beta:.2f}
R-squared: {capm_model.rsquared:.3f}
p-value (Beta): {capm_model.pvalues[1]:.3g}
Number of observations (N): {int(capm_model.nobs)}
Standard error (Beta): {capm_model.bse[1]:.3f}
"""

print(table_text)
print(capm_model.summary())
```

```
import pandas as pd
import numpy as np
```

```
import yfinance as yf
import matplotlib.pyplot as plt
import statsmodels.api as sm
from datetime import datetime

# Plot settings
plt.style.use("default")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["axes.grid"] = True

# Parameters
TICKER = "NVDA"     # NVIDIA
BENCHMARK = "^GSPC"  # S&P 500 (you can change to ^NDX etc.)
START_DATE = "2015-01-01"
END_DATE = datetime.today().strftime("%Y-%m-%d")

print("Settings OK:", TICKER, BENCHMARK, START_DATE, END_DATE)
```

```
nvda = yf.download(TICKER, start=START_DATE, end=END_DATE)
bench = yf.download(BENCHMARK, start=START_DATE, end=END_DATE)

print("NVDA columns:", nvda.columns.tolist())
print("Benchmark columns:", bench.columns.tolist())
nvda.head()
```

```
def get_price_column(df):
    """Return the best available price column name."""
    for col in ["Adj Close", "Adj_Close", "Close", "close"]:
        if col in df.columns:
            return col
    raise KeyError("No suitable price column found (Adj Close / 
Close).")

price_col_nvda = get_price_column(nvda)
price_col_bench = get_price_column(bench)

print("Using NVDA price column:", price_col_nvda)
print("Using benchmark price column:", price_col_bench)

# Daily returns
nvda["Return"] = nvda[price_col_nvda].pct_change()
bench["Return"] = bench[price_col_bench].pct_change()

# Combined returns DataFrame
returns = pd.concat(
    [nvda["Return"], bench["Return"]],
    axis=1,
    keys=[TICKER, "Market"]
```

```
).dropna()

returns.head()
```

```
nvda["MA50"] = nvda[price_col_nvda].rolling(window=50).mean()
nvda["MA200"] = nvda[price_col_nvda].rolling(window=200).mean()

fig, ax = plt.subplots()
ax.plot(nvda.index, nvda[price_col_nvda], label=f"{TICKER} price")
ax.plot(nvda.index, nvda["MA50"], label="50-day MA")
ax.plot(nvda.index, nvda["MA200"], label="200-day MA")

ax.set_title(
    f"{TICKER} Price with 50- and 200-day Moving Averages\n"
    f"Source: Yahoo Finance, currency: USD, period: {START_DATE}–
{END_DATE}"
)
ax.set_xlabel("Date")
ax.set_ylabel("Price (USD)")
ax.legend()
plt.tight_layout()
plt.show()
```

```
nvda["MB"] = nvda[price_col_nvda].rolling(window=window).mean()   #
Middle band
nvda["STD"] = nvda[price_col_nvda].rolling(window=window).std()
nvda["UB"] = nvda["MB"] + 2 * nvda["STD"]                         #
Upper band
nvda["LB"] = nvda["MB"] - 2 * nvda["STD"]                         #
Lower band
```

```
fig, ax = plt.subplots()
ax.plot(nvda.index, nvda[price_col_nvda], label=f"{TICKER} price")
ax.plot(nvda.index, nvda["MB"], label="Middle Band (20-day MA)")
ax.plot(nvda.index, nvda["UB"], label="Upper Band (+2σ)")
ax.plot(nvda.index, nvda["LB"], label="Lower Band (-2σ)")
```

```
ax.set_title(
    f"{TICKER} Bollinger Bands (20 days)\n"
    f"Source: Yahoo Finance, currency: USD, period: {START_DATE}–
{END_DATE}"
)
ax.set_xlabel("Date")
ax.set_ylabel("Price (USD)")
ax.legend()
plt.tight_layout()
plt.show()
```

```
def compute_rsi(series: pd.Series, period: int = 14) -> pd.Series:
```

```
    """Compute Relative Strength Index (RSI)."""
    delta = series.diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)

    avg_gain = gain.rolling(window=period, min_periods=period).mean()
    avg_loss = loss.rolling(window=period, min_periods=period).mean()

    rs = avg_gain / avg_loss
    rsi = 100 - (100 / (1 + rs))
    return rsi

nvda["RSI14"] = compute_rsi(nvda[price_col_nvda], period=14)

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(10, 8))

ax1.plot(nvda.index, nvda[price_col_nvda], label=f"{TICKER} price")
ax1.set_ylabel("Price (USD)")
ax1.legend()

ax2.plot(nvda.index, nvda["RSI14"], label="RSI (14 days)")
ax2.axhline(70, linestyle="--")
ax2.axhline(30, linestyle="--")
ax2.set_ylabel("RSI")
ax2.set_xlabel("Date")
ax2.legend()

fig.suptitle(
    f"{TICKER} Price and 14-day RSI\n"
    f"Source: Yahoo Finance, currency: USD, period: {START_DATE}–
{END_DATE}"
)
plt.tight_layout()
plt.show()
```

```
annual_rf = 0.04
daily_rf = (1 + annual_rf) ** (1/252) - 1

returns["Rf"] = daily_rf
returns["Excess_NVDA"] = returns[TICKER] - returns["Rf"]
returns["Excess_Market"] = returns["Market"] - returns["Rf"]

returns.head()

capm_data = returns[["Excess_NVDA", "Excess_Market"]].dropna()

Y = capm_data["Excess_NVDA"]
X = sm.add_constant(capm_data["Excess_Market"])
```

```
capm_model = sm.OLS(Y, X).fit()
print(capm_model.summary())
alpha = capm_model.params["const"]
beta = capm_model.params["Excess_Market"]

fig, ax = plt.subplots()
ax.scatter(
    capm_data["Excess_Market"], capm_data["Excess_NVDA"],
    s=5, alpha=0.5, label="Daily observations"
)

x_vals = np.linspace(capm_data["Excess_Market"].min(),
                     capm_data["Excess_Market"].max(), 100)
y_vals = alpha + beta * x_vals
ax.plot(x_vals, y_vals, label=f"CAPM line (alpha={alpha:.4f},
beta={beta:.2f})")

ax.set_title(
    f"CAPM Regression: {TICKER} vs {BENCHMARK}\n"
    f"Source: Yahoo Finance, daily excess returns, period:
{START_DATE}–{END_DATE}"
)
ax.set_xlabel("Market excess return")
ax.set_ylabel(f"{TICKER} excess return")
ax.legend()
plt.tight_layout()
plt.show()
```

```
cum_nvda = (1 + returns[TICKER]).cumprod()
cum_market = (1 + returns["Market"]).cumprod()

fig, ax = plt.subplots()
ax.plot(cum_nvda.index, cum_nvda, label=TICKER)
ax.plot(cum_market.index, cum_market, label="Market")

ax.set_title(
    f"Cumulative Returns: {TICKER} vs {BENCHMARK}\n"
    f"Source: Yahoo Finance, currency: USD, period: {START_DATE}–
{END_DATE}"
)
ax.set_xlabel("Date")
ax.set_ylabel("Growth of $1 investment")
ax.legend()
plt.tight_layout()
plt.show()
```

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
nvda_ticker = yf.Ticker("NVDA")

fin = nvda_ticker.financials
bs = nvda_ticker.balance_sheet
cf = nvda_ticker.cashflow

print("Income statement rows:", fin.index.tolist())
print("Balance sheet rows:", bs.index.tolist())
print("Cash flow rows:", cf.index.tolist())
print("Columns (fiscal period ends):", fin.columns)
```

```
def get_row(df, row_name):
    if row_name in df.index:
        return df.loc[row_name]
    else:
        return pd.Series(index=df.columns, dtype="float64")

fund_raw = pd.DataFrame({
    "Revenue":          get_row(fin, "Total Revenue"),
    "NetIncome":        get_row(fin, "Net Income"),
    "OperatingIncome":  get_row(fin, "Operating Income"),
    "InterestExpense":  get_row(fin, "Interest Expense"),
    "TotalAssets":      get_row(bs, "Total Assets"),
    "TotalEquity":      get_row(bs, "Total Stockholder Equity"),
    "TotalDebt":        get_row(bs, "Total Debt"),
    "CurrentAssets":    get_row(bs, "Total Current Assets"),
    "CurrentLiabilities": get_row(bs, "Total Current Liabilities"),
    "OperatingCashFlow":  get_row(cf, "Operating Cash Flow")
})

fund = fund_raw.T.copy()
fund.columns = [c.year for c in fund.columns]
fund = fund.T.sort_index()
```

```
fund["NetMargin"] = fund["NetIncome"] / fund["Revenue"]
fund["ROA"] = fund["NetIncome"] / fund["TotalAssets"]
fund["ROE"] = fund["NetIncome"] / fund["TotalEquity"]

fund["DebtToEquity"] = fund["TotalDebt"] / fund["TotalEquity"]
fund["DebtToAssets"] = fund["TotalDebt"] / fund["TotalAssets"]
fund["InterestCoverage"] = fund["OperatingIncome"] / (-
fund["InterestExpense"].replace(0, np.nan))
```

```
fund["CurrentRatio"] = fund["CurrentAssets"] /
fund["CurrentLiabilities"]
fund["OCF_to_NI"] = fund["OperatingCashFlow"] / fund["NetIncome"]

fund["RevenueGrowth"] = fund["Revenue"].pct_change()
fund["NetIncomeGrowth"] = fund["NetIncome"].pct_change()

fund
```

```
fig, ax = plt.subplots()

ax.plot(fund.index, fund["NetMargin"] * 100, marker="o", label="Net
margin (%)")
ax.plot(fund.index, fund["ROA"] * 100, marker="o", label="ROA (%)")
ax.plot(fund.index, fund["ROE"] * 100, marker="o", label="ROE (%)")

ax.set_title(
    "NVIDIA Profitability Ratios\n"
    "Source: Yahoo Finance (NVDA financials), currency: USD"
)
ax.set_xlabel("Fiscal year")
ax.set_ylabel("Ratio (%)")
ax.legend()
plt.tight_layout()
plt.show()
```

```
fig, ax1 = plt.subplots()

ax1.plot(fund.index, fund["Revenue"] / 1e9, marker="o", label="Revenue
(USD bn)")
ax1.plot(fund.index, fund["NetIncome"] / 1e9, marker="o", label="Net
income (USD bn)")

ax1.set_title(
    "NVIDIA Revenue and Net Income Growth\n"
    "Source: Yahoo Finance (NVDA financials), currency: USD"
)
ax1.set_xlabel("Fiscal year")
ax1.set_ylabel("USD billions")
ax1.legend()
plt.tight_layout()
plt.show()
```

```
fig, ax = plt.subplots()
```

```
ax.plot(fund.index, fund["DebtToEquity"], marker="o",
label="Debt/Equity")
ax.plot(fund.index, fund["DebtToAssets"], marker="o",
label="Debt/Assets")
ax.plot(fund.index, fund["InterestCoverage"], marker="o",
label="Interest coverage (x)")

ax.set_title(
    "NVIDIA Capital Structure and Solvency Ratios\n"
    "Source: Yahoo Finance (NVDA financials & balance sheet), currency:
USD"
)
ax.set_xlabel("Fiscal year")
ax.set_ylabel("Ratio")
ax.legend()
plt.tight_layout()
plt.show()
fig, ax = plt.subplots()

ax.plot(fund.index, fund["CurrentRatio"], marker="o", label="Current
ratio")
ax.plot(fund.index, fund["OCF_to_NI"], marker="o", label="Operating
cash flow / Net income")

ax.set_title(
    "NVIDIA Liquidity and Cash-Flow Quality\n"
    "Source: Yahoo Finance (NVDA financials, balance sheet, cash flow),
currency: USD"
)
ax.set_xlabel("Fiscal year")
ax.set_ylabel("Ratio")
ax.legend()
plt.tight_layout()
plt.show()
```

```
peer_tickers = ["NVDA", "AMD", "AVGO"]
peers = {t: yf.Ticker(t) for t in peer_tickers}

data = []
for t, tk in peers.items():
    info = tk.info  # may be slow; needs internet
    data.append({
        "Ticker": t,
        "PE (TTM)": info.get("trailingPE", np.nan),
        "Forward P/E": info.get("forwardPE", np.nan),
        "P/S (TTM)": info.get("priceToSalesTrailing12Months", np.nan),
        "P/B": info.get("priceToBook", np.nan),
        "EV/EBITDA": info.get("enterpriseToEbitda", np.nan)
```

```
    })

val_df = pd.DataFrame(data).set_index("Ticker")
val_df
fig, ax = plt.subplots()

val_df[["PE (TTM)", "EV/EBITDA"]].plot(kind="bar", ax=ax)

ax.set_title(
    "Valuation Multiples: NVIDIA vs Peers (AMD, AVGO)\n"
    "Source: Yahoo Finance, currency: USD"
)
ax.set_xlabel("Ticker")
ax.set_ylabel("Multiple")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

tickers = ["NVDA", "AMD", "AVGO"]

rows = []
for t in tickers:
    info = yf.Ticker(t).info
    rows.append({
        "Ticker": t,
        "MarketCap (USD bn)": info.get("marketCap", np.nan) / 1e9,
        "P/E (TTM)": info.get("trailingPE", np.nan),
        "ROE (%)": (info.get("returnOnEquity", np.nan) or np.nan) * 100
    })

comp_df = pd.DataFrame(rows).set_index("Ticker")
comp_df
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

comp_df["MarketCap (USD bn)"].plot(kind="bar", ax=axes[0])
axes[0].set_title("Market capitalisation")
axes[0].set_ylabel("USD billions")

comp_df["P/E (TTM)"].plot(kind="bar", ax=axes[1])
axes[1].set_title("P/E ratio (TTM)")
```

```
comp_df["ROE (%)"].plot(kind="bar", ax=axes[2])
axes[2].set_title("Return on equity")
axes[2].set_ylabel("%")

for ax in axes:
    ax.set_xlabel("Ticker")
    ax.tick_params(axis="x", rotation=0)

fig.suptitle(
    "NVIDIA vs peers: Market cap, P/E and ROE\n"
    "Source: Yahoo Finance, currency: USD",
    y=1.05
)
plt.tight_layout()
plt.show()
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Ticker": ["NVDA", "AMD", "AVGO"],
    "MarketCap (USD bn)": [325.97, 360.81, 188.90],
    "P/E (TTM)": [36.8, 46.8, 106.4],
    "ROE (%)": [31.4, 105.3, 28.7]
}

comp_df = pd.DataFrame(data).set_index("Ticker")
```

```
import pandas as pd
import matplotlib.pyplot as plt

# example – change numbers to yours if you enter them manually
data = {
    "Ticker": ["NVDA", "AMD", "AVGO"],
    "MarketCap (USD bn)": [325.97, 360.81, 188.90],
    "P/E (TTM)": [36.8, 46.8, 106.4],
    "ROE (%)": [31.4, 105.3, 28.7]
}

comp_df = pd.DataFrame(data).set_index("Ticker")
metrics = ["MarketCap (USD bn)", "P/E (TTM)", "ROE (%)"]

ax = comp_df[metrics].T.plot(
    kind="bar",
    figsize=(9, 5)
)
```

```
ax.set_title("Comparison of NVDA, AMD and AVGO\nby Market cap, P/E and
ROE")
ax.set_xlabel("Metric")
ax.set_ylabel("Market cap (USD bn) / Ratios")
ax.legend(title="Ticker")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
import matplotlib.pyplot as plt
import pandas as pd

years = [2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026]


data_center = [1.932, 2.932, 2.983, 6.696, 10.613, 15.005, 47.525,
115.186, 191.661]
gaming      = [5.513, 6.246, 5.518, 7.759, 12.462, 9.067, 10.447,
11.350, 16.355]
auto        = [0.558, 0.641, 0.700, 0.536, 0.566, 0.903, 1.091, 1.694,
2.392]
pro_viz     = [0.934, 1.060, 1.053, 1.053, 1.211, 1.544, 0.0, 0.0, 0.0]

df = pd.DataFrame({
    "Year": years,
    "Data Center": data_center,
    "Gaming": gaming,
    "Automotive": auto,

})

df
plt.figure(figsize=(10, 6))

plt.plot(df["Year"], df["Data Center"], marker="o", label="Data
Center")
plt.plot(df["Year"], df["Gaming"], marker="o", label="Gaming")
plt.plot(df["Year"], df["Automotive"], marker="o", label="Automotive")

plt.title(
    "NVIDIA Revenue by Segment\n"
    "Source: Bloomberg, currency: USD (billions)"
)
plt.xlabel("Fiscal year")
plt.ylabel("Revenue (USD billions)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
```

```
plt.show()
```

```
!pip install yfinance --quiet

import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.callbacks import EarlyStopping

np.random.seed(42)

ticker = "NVDA"
data = yf.download(ticker, start="2015-01-01")

try:

    df = data[[('Close', ticker)]].copy()
    df.columns = ['Adj Close'] # Rename the column to 'Adj Close'
except KeyError:

    if 'Adj Close' in data.columns:
        df = data[["Adj Close"]].copy()
    elif 'Close' in data.columns:
        df = data[["Close"]].copy()
        df.columns = ['Adj Close'] # Rename for consistency
    else:
        raise KeyError(f"Neither 'Adj Close' nor 'Close' column found
in the downloaded data for {ticker}.")

df.dropna(inplace=True)

plt.figure(figsize=(10,4))
plt.plot(df.index, df["Adj Close"])
plt.title("NVDA Adjusted Close (historical)")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.grid(True)
plt.show()

scaler = MinMaxScaler(feature_range=(0,1))
scaled_prices = scaler.fit_transform(df[["Adj Close"]])

def create_sequences(series, seq_len=60):
```

```
    X, y = [], []
    for i in range(seq_len, len(series)):
        X.append(series[i-seq_len:i, 0])
        y.append(series[i, 0])
    return np.array(X), np.array(y)

seq_len = 60
X, y = create_sequences(scaled_prices, seq_len=seq_len)

X = X.reshape((X.shape[0], X.shape[1], 1))

split = int(len(X) * 0.85)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

model = Sequential()
model.add(LSTM(64, return_sequences=True, input_shape=(seq_len, 1)))
model.add(LSTM(64))
model.add(Dense(1))

model.compile(optimizer="adam", loss="mean_squared_error")

es = EarlyStopping(monitor="val_loss", patience=5,
restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=40,
    batch_size=32,
    validation_data=(X_test, y_test),
    callbacks=[es],
    verbose=1
)

all_pred_scaled = model.predict(X)
all_pred = scaler.inverse_transform(all_pred_scaled.reshape(-1,1))

pred_index = df.index[seq_len:]

df_pred = pd.DataFrame(
    {
        "Actual": df["Adj Close"].values[seq_len:],
        "LSTM_pred": all_pred.flatten()
    },
    index=pred_index
)

plt.figure(figsize=(12,5))
```

```
plt.plot(df_pred.index, df_pred["Actual"], label="Actual price")
plt.plot(df_pred.index, df_pred["LSTM_pred"], label="LSTM fitted",
alpha=0.8)
plt.title("NVDA – LSTM fitted vs actual (historical)")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

future_steps = 504

last_sequence = scaled_prices[-seq_len:].reshape(1, seq_len, 1)

future_scaled = []

for _ in range(future_steps):
    next_scaled = model.predict(last_sequence, verbose=0)[0,0]
    future_scaled.append(next_scaled)

    new_seq = np.append(last_sequence[0,1:,0], next_scaled)
    last_sequence = new_seq.reshape(1, seq_len, 1)

future_scaled = np.array(future_scaled).reshape(-1,1)
future_prices = scaler.inverse_transform(future_scaled).flatten()

last_date = df.index[-1]
future_dates = pd.date_range(start=last_date + pd.Timedelta(days=1),
                             periods=future_steps,
                             freq="B")

df_future = pd.DataFrame(
    {"Forecast_price": future_prices},
    index=future_dates
)

plt.figure(figsize=(14,6))

plt.plot(df.index, df["Adj Close"], label="Historical price")

plt.plot(df_future.index, df_future["Forecast_price"], "r--",
         label="LSTM forecast (next ~2 years)")

plt.title("NVDA – LSTM share price forecast\n(All data & model purely
illustrative)")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
```

```
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print(f"Predicted NVDA price on {df_future.index[-1].date()}: "
      f"${df_future['Forecast_price'].iloc[-1]:.2f}")
```

```
!pip install yfinance --quiet

import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.callbacks import EarlyStopping

np.random.seed(42)

ticker = "NVDA"

data = yf.download(ticker, start="2020-01-01", auto_adjust=False)

if "Adj Close" in data.columns:
    price_col_name = "Adj Close"
else:
    price_col_name = "Close"

df = data[[price_col_name]].copy()
df.dropna(inplace=True)

scaler = MinMaxScaler(feature_range=(0, 1))
scaled_prices = scaler.fit_transform(df[[price_col_name]])

def create_sequences(series, seq_len=60):
    X, y = [], []
    for i in range(seq_len, len(series)):
        X.append(series[i-seq_len:i, 0])
        y.append(series[i, 0])
    return np.array(X), np.array(y)

seq_len = 60
X, y = create_sequences(scaled_prices, seq_len=seq_len)

X = X.reshape((X.shape[0], X.shape[1], 1))
```

```
split = int(len(X) * 0.85)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

model = Sequential()
model.add(LSTM(64, return_sequences=True, input_shape=(seq_len, 1)))
model.add(LSTM(64))
model.add(Dense(1))

model.compile(optimizer="adam", loss="mean_squared_error")

es = EarlyStopping(monitor="val_loss", patience=5,
restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=40,
    batch_size=32,
    validation_data=(X_test, y_test),
    callbacks=[es],
    verbose=1
)

all_pred_scaled = model.predict(X)
all_pred = scaler.inverse_transform(all_pred_scaled.reshape(-1, 1))

pred_index = df.index[seq_len:]

df_pred = pd.DataFrame(
    {
        "Actual": df[price_col_name].squeeze().values[seq_len:],
        "LSTM_pred": all_pred.flatten()
    },
    index=pred_index
)

plt.figure(figsize=(12, 5))
plt.plot(df_pred.index, df_pred["Actual"], label="Actual price")
plt.plot(df_pred.index, df_pred["LSTM_pred"], label="LSTM fitted",
alpha=0.8)
plt.title("NVDA – LSTM fitted vs actual (2020–present)")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
future_steps = 378

last_sequence = scaled_prices[-seq_len:].reshape(1, seq_len, 1)
future_scaled = []

for _ in range(future_steps):
    next_scaled = model.predict(last_sequence, verbose=0)[0, 0]
    future_scaled.append(next_scaled)


    new_seq = np.append(last_sequence[0, 1:, 0], next_scaled)
    last_sequence = new_seq.reshape(1, seq_len, 1)

future_scaled = np.array(future_scaled).reshape(-1, 1)
future_prices = scaler.inverse_transform(future_scaled).flatten()

last_date = df.index[-1]
future_dates = pd.date_range(start=last_date + pd.Timedelta(days=1),
                             periods=future_steps,
                             freq="B")

df_future = pd.DataFrame({"Forecast_price": future_prices},
index=future_dates)

history_window = 252 * 2
hist_prices = df[price_col_name].squeeze().iloc[-history_window:]
hist_dates = df.index[-history_window:]

plt.figure(figsize=(12, 5))
plt.plot(hist_dates, hist_prices, label="Historical price (last ~2
years)")
plt.plot(df_future.index, df_future["Forecast_price"], "r--",
         label="LSTM forecast (next ~1.5 years)")

plt.title("NVDA – LSTM share price prediction\n(illustrative machine-
learning model)")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print(
    f"Predicted NVDA price on {df_future.index[-1].date()}: "
    f"${df_future['Forecast_price'].iloc[-1]:.2f}"
)
```